A Lightweight Two-Node High-Availability LAMP Stack: Design, Implementation, and Failover Evaluation with Open-Source Tools

1st Raj Dangi
University of Toledo, College of Engineering
Toledo, USA
rdangi@rockets.utoledo.edu

***Abstract*—High availability is typically achieved through dedicated cloud infrastructure, hardware redundancy, or specialized orchestration frameworks—resources often inaccessible to students or small organizations. This work implements a two-node fault tolerant web system using commonly available components such as Linux, Apache, MariaDB, and PHP (LAMP) running across a primary node and synchronized backup node connected through an encrypted overlay network. Unlike conventional cloud-based approaches, the architecture operates entirely on commodity hardware. A load balancer (HAProxy) continuously monitors backend health and redirects traffic when node failure occurs, while automated synchronization ensures consistency of application state and database contents. Experimental results demonstrate a failover time of approximately 3 seconds, failback recovery of 5 seconds, and negligible changes in request latency (0.010 seconds on the primary node versus 0.012 seconds on the backup node). These findings suggest that meaningful availability guarantees can be achieved using lightweight open-source tooling without cloud services or cluster frameworks.**

***Index Terms*—High availability, web infrastructure, load balancing, overlay networking, HAProxy, Tailscale, LAMP**

I. Introduction

Conventional web hosting architectures operate on a single server instance, meaning service availability is tightly coupled to the health of one machine. Total service disruption can occur due to hardware faults, software misconfiguration, human error, or network outage. High availability systems mitigate these risks through redundancy and automated routing, but traditional implementations rely on enterprise tooling or cloud platforms.

This work explores whether meaningful resilience can be achieved with commodity hardware available to students. The system consists of a primary server running Arch Linux and a synchronized Ubuntu-based backup running in a virtual machine. Both nodes run the Linux, Apache, MariaDB, and PHP (LAMP) stack. A lightweight encrypted overlay network (Tailscale) enables peer-to-peer communication between nodes without exposing services publicly or requiring conventional VPN configuration.

The research question addressed is: *Can student-scale infrastructure achieve measurable availability improvements using open-source tools without enterprise clustering or cloud-native services?*

II. System Architecture

A. Component Overview

The completed system comprises:

- A primary web server running on Arch Linux
- A virtualized backup server running Ubuntu Server
- HAProxy load balancer monitoring both nodes
- Secure communication via Tailscale private networking
- Web service stack using Apache, PHP, and MariaDB
- Automatic file and database synchronization via cron automation

B. Communication Model

Client traffic is routed through HAProxy configured on the primary machine. HAProxy polls both backends using health-check endpoints. If the primary node becomes unavailable, HAProxy transparently forwards requests to the backup. When the primary recovers, service is restored without intervention.

C. Data Consistency Model

The system implements eventual consistency. File changes propagate using Remote Sync (rsync) every five minutes, and database changes replicate using periodic mysqldump export and import every ten minutes. Although this approach is not real-time, it supports applications tolerant of short propagation delays.

D. Node Identity Verification

Figures 1 and 2 illustrate distinct identity responses returned by the primary and backup nodes, used during failover evaluation.

III. Implementation

A. Primary Node

The primary node runs Apache on localhost port 8080 while HAProxy binds externally to port 80. The web application is deployed from a Git-backed directory, and health endpoints expose service status.

B. Backup Node

The backup node replicates the full stack with differences in node identity and routing logic. Deployment occurs through automated file transfer over encrypted transport.

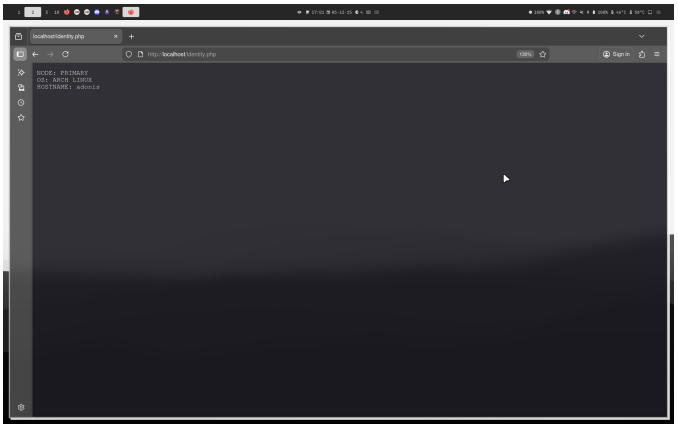

Fig. 1. Primary node (Arch Linux) identity endpoint confirming active backend routing.

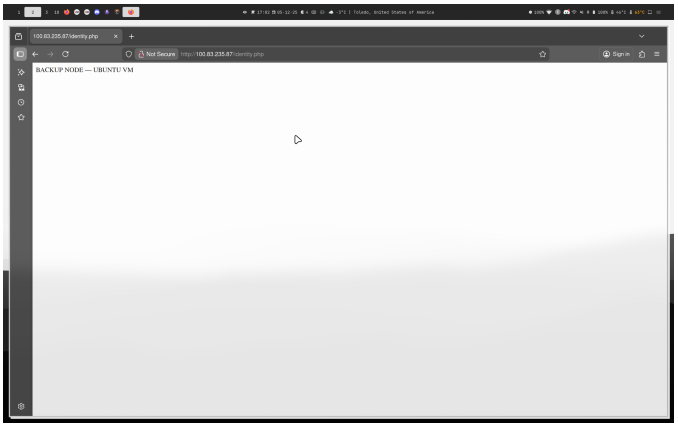

Fig. 2. Backup node (Ubuntu VM) identity endpoint verifying secondary failover capability.

C. Overlay Network

Tailscale was chosen over conventional VPNs for ease of deployment, automatic authentication, peer-to-peer routing, and reduced configuration complexity. This provides secure connectivity without exposing ports publicly.

D. Load Balancer Configuration

HAProxy performs health polling at three-second intervals with two-failure demotion and two-success promotion. This results in a measured failover interval of approximately three seconds and failback of five seconds. Figure 3 displays the configuration used.

E. Synchronization Automation

Two automation scripts execute under cron scheduling:

- File synchronization every five minutes
- Database synchronization every ten minutes

Figures 4 and 5 show logged evidence of synchronization activity.

Fig. 3. HAProxy configuration implementing backend health checks and failover policy.

Fig. 4. File synchronization logs on the primary node showing periodic propagation to the backup.

Fig. 5. Database synchronization logs demonstrating mysqldump export and restore behavior.

IV. Evaluation

Fault injection was performed by shutting down Apache on the primary node while observing service continuity through web requests. Figure 6 illustrates that HAProxy reroutes traffic to the backup node immediately after primary failure.

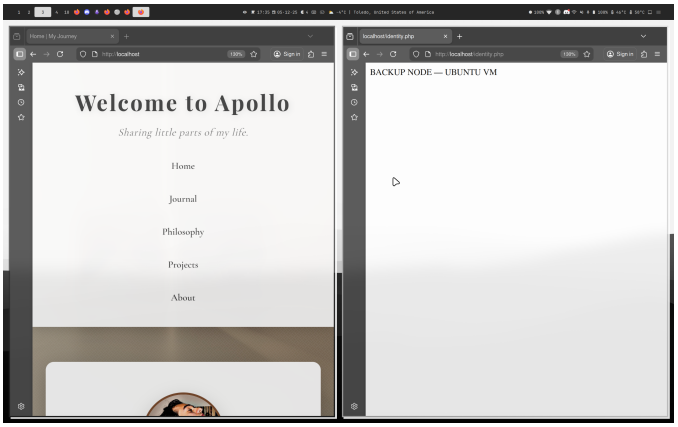

Fig. 6. Failover demonstration: routing automatically shifts to backup node after primary shutdown, reflected in identity view and website response.

Upon restoration of the primary node, routing returns to it, as shown in Figure 7.

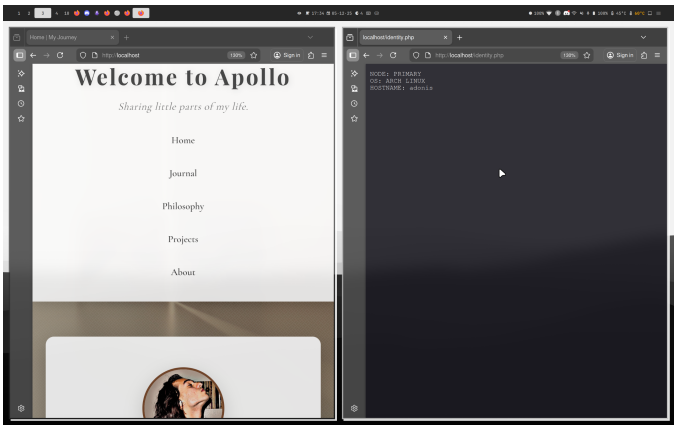

Fig. 7. Failback demonstration: HAProxy restores routing to the primary node once it recovers and resumes service.

Response latency on the primary node averaged 0.010 seconds compared to 0.012 seconds on the backup node, representing negligible degradation. File modifications became visible on the backup within the five-minute synchronization window, and database values propagated within ten minutes.

V. Discussion

The presented system achieves student-scale fault tolerance with low cost and moderate implementation effort. Limitations include eventual consistency, no split-brain prevention, and lack of real-time replication. Future enhancements may include distributed consensus protocols, dual-primary replication, floating IP failover, or continuous synchronization triggered by filesystem events rather than static intervals.

VI. Conclusion

This study demonstrates that measurable fault tolerance, automated routing, and resilience behavior can be achieved on low-resource infrastructure using widely available tools. The two-node architecture successfully rerouted traffic during failure events with minimal observable latency change and recovered operation autonomously. These findings suggest that educational environments can model availability principles effectively without enterprise infrastructure.

Acknowledgment

The author would like to thank Anushka Sharad Kumavat (askumavat@owu.edu) for her valuable feedback on the structure, clarity, and presentation of this research. Her suggestions significantly improved the academic rigor and organization of this work.

The author also acknowledges Purshottam Singh Thakur and Jay Doshi for their collaborative involvement during earlier versions of the system build and experimentation process. Their assistance was invaluable in refining implementation decisions and validating operational behavior.

References

[1] W. Tarreau, “HAProxy - The Reliable, High Performance TCP/HTTP Load Balancer,” [Online]. Available: https://www.haproxy.org/

[2] A. Tridgell and P. Mackerras, “The rsync algorithm,” Australian National University, Dept. Computer Science, Tech. Rep. TR-CS-96-05, 1996.

[3] A. Pennarun, “How Tailscale works,” Tailscale Blog, Mar. 2020. [Online]. Available: https://tailscale.com/blog/how-tailscale-works/

[4] MariaDB Foundation, “MariaDB Server Documentation,” 2024. [Online]. Available: https://mariadb.com/kb/en/

[5] S. S. Gill, A. K. Luhach, and N. Chilamkurti, “Design and Implementation of High-Availability Architecture for IoT-Cloud Services,” in *IEEE Internet of Things Journal*, vol. 6, no. 6, pp. 10868-10879, Dec. 2019.

[6] A. A. Saif and M. Shafi’i M. A., “A Survey of Dynamic Replication Strategies in Distributed Systems,” in *International Journal of Computer Applications*, vol. 110, no. 11, pp. 1-4, 2015.

[7] P. Vixie, “Cron,” in *ISC Cron Documentation*, 1993. [Online]. Available: https://ftp.isc.org/isc/cron/

[8] K. Sibbald, “Bacula Main Reference Manual,” Bacula.org, Mar. 2017. [Online]. Available: https://www.bacula.org/7.4.x-manuals/en/main/main.pdf

[9] M. Welsh, D. Culler, and E. Brewer, “SEDA: An architecture for well-conditioned, scalable internet services,” in *Proc. 18th ACM Symposium on Operating Systems Principles (SOSP)*, pp. 230-243, 2001.